

Practical Lab: Lab 5 — ARP Poisoning Forensics

Student name: Afeez Bolaji Usman

Student ID: 202514326WY

Course Code: SBT-DF203

Course Title : Basic Networking Skills for Digital Forensics

Lab Title: Lab 5 — ARP Poisoning Forensics

Instructor Name: Aminu Idris

Date: 17/9/2026

1. Executive Summary and Authorization

This report documents a forensic examination of ARP resolution behaviour performed under the SBT-DF203 Lab 5 authorization. All activity was confined to an isolated Kali Linux VM and the ICDDFA-supplied training capture (arp.pcap). No live ARP poisoning simulation was performed because instructor approval for the optional Part D component was not obtained.

Two evidence sources were analyzed: (1) a live capture of normal ARP resolution against the trainee's own default gateway and (2) the supplied arp.pcap training capture, which was examined for conflicting IP-to-MAC claims and other indicators of ARP poisoning.

Normal ARP resolution on the trainee's VM behaved as expected, consisting of a broadcast ARP request followed by a unicast ARP reply. The default gateway consistently resolves to MAC address 00:50:56:f2:0e:73.

Analysis of the supplied arp.pcap did not identify a classic same-IP/multiple-MAC conflict. However, one packet-level anomaly was observed: frame 5 contains a unicast (non-broadcast)

ARP request, which deviates from the expected normal ARP request pattern described in RFC 826. This is documented as an observed anomaly that may be consistent with a targeted or crafted ARP interaction. It is not treated as confirmed ARP poisoning, because the capture does not provide a corresponding dual-MAC conflict or other corroborating evidence of an active man-in-the-middle condition.

2. Network Diagram and Assigned IP/MAC Addresses

Analysis environment

Role	IP Address	Mac Address	Note
Analyst VM (trainee)	192.168.159.128	192.168.159.128	Kali Linux VM, eth0
Default gateway	192.168.159.2	00:50:56:f2:0e:73 (VMware_f2:0e:73)	Confirmed legitimate MAC via live ARP resolution
Other cached host	192.168.159.254	00:50:56:ec:66:07	Present in initial ARP cache, not otherwise queried

Supplied evidence

Role	IP Address	Mac Address	Note
Router/gateway device	136.160.215.1	00:1b:17:00:0a:30 (Dell)	Periodic gratuitous selfannounce + duplicate address probes
Host A	136.160.215.194	00:50:56:86:02:65 (VMware)	Initiated anomalous unicast request (frame 5)
Host B	136.160.215.15	00:50:56:86:cb:fc (VMware)	Target of frame 5's nonbroadcast request

3. Evidence Integrity

The original evidence file (arp.pcap) was preserved and a verified working copy was created before analysis

began. SHA-256 hashes of both files match, confirming byte-for-byte integrity.

File	SHA-256
evidence/arp.pcap (original)	342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e
working/arp_working.pcap (working copy)	342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e

```
(kali㉿kali)-[~/SBT-DF203-Lab5]
$ cp --preserve=timestamps evidence/arp.pcap working/arp_working.pcap

(kali㉿kali)-[~/SBT-DF203-Lab5]
$ sha256sum evidence/arp.pcap working/arp_working.pcap | tee reports/arp_capture_hashes.txt
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e  evidence/arp.pcap
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e  working/arp_working.pcap
```

```
(kali㉿kali)-[~/SBT-DF203-Lab5]
$ ip -br address | tee reports/interfaces.txt
lo                UNKNOWN          127.0.0.1/8  ::1/128
eth0              UP                192.168.159.128/24  fe80::bd40:b67:d601:4a1/64
docker0          DOWN             172.17.0.1/16

(kali㉿kali)-[~/SBT-DF203-Lab5]
$ ip route | tee reports/routes.txt
default via 192.168.159.2 dev eth0 proto dhcp src 192.168.159.128 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
192.168.159.0/24 dev eth0 proto kernel scope link src 192.168.159.128 metric 100

(kali㉿kali)-[~/SBT-DF203-Lab5]
$ ip neigh show | tee reports/arp_table_initial.txt
192.168.159.254 dev eth0 lladdr 00:50:56:ec:66:07 STALE
192.168.159.2 dev eth0 lladdr 00:50:56:f2:0e:73 STALE
```

Chain-of-custody worksheet

Field	Entry
-------	-------

Case/lab identifier	SBT-DF203-Lab5-Afeez Bolaji Usman
Trainee name	Afeez Bolaji
Date and time started	17-09-2026
Evidence file name(s)	arp.pcap
Source or generation method	ICDFA-supplied training capture
Original SHA-256	342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e
Working-copy SHA-256	342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e
Analysis workstation/VM	Kali Linux (kali@kali)
Notes on any changes	None — working copy verified identical to original

Part A

```
(kali㉿kali)-[~/SBT-DF203-Lab5]
└─$ sudo ip neigh flush 192.168.159.2 dev eth0
sudo tshark -i eth0 -f 'arp' -a duration:15 -w /tmp/normal_arp.pcapng &

sleep 2
ping -c 1 192.168.159.2
wait
[1] 42806
Running as user "root" and group "root". This could be dangerous.
Capturing on 'eth0'
PING 192.168.159.2 (192.168.159.2) 56(84) bytes of data.
64 bytes from 192.168.159.2: icmp_seq=1 ttl=128 time=0.447 ms

— 192.168.159.2 ping statistics —
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.447/0.447/0.447/0.000 ms
2
[1] + done      sudo tshark -i eth0 -f 'arp' -a duration:15 -w /tmp/n
ormal_arp.pcapng
```

```
(kali㉿kali)-[~/SBT-DF203-Lab5]
└─$ capinfos /tmp/normal_arp.pcapng
sudo tshark -r /tmp/normal_arp.pcapng
capinfos: You don't have permission to read the file "/tmp/normal_arp.p
capng".
Running as user "root" and group "root". This could be dangerous.
  1 0.0000000000 VMware_0e:d1:ce → Broadcast      ARP 42 Who has 192.168
.159.2? Tell 192.168.159.128
  2 0.000222272 VMware_f2:0e:73 → VMware_0e:d1:ce ARP 60 192.168.159.
2 is at 00:50:56:f2:0e:73
```

Part B

```
(kali㉿kali)-[~/SBT-DF203-Lab5]
$ tshark -r evidence/normal_arp.pcapng -Y 'arp' -T fields \
-e frame.number -e frame.time -e eth.src -e eth.dst -e arp.opcode \
-e arp.src.proto_ipv4 -e arp.src.hw_mac -e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
| tee reports/normal_arp_fields.tsv
1      2026-09-16T22:15:47.629874475-0400      00:0c:29:0e:d1:ce      ff:ff:ff:f
f:ff:ff 1      192.168.159.128 00:0c:29:0e:d1:ce      192.168.159.2  00:00:00:0
0:00:00
2      2026-09-16T22:15:47.630096747-0400      00:50:56:f2:0e:73      00:0c:29:0
e:d1:ce 2      192.168.159.2  00:50:56:f2:0e:73      192.168.159.128 00:0c:29:0
e:d1:ce
```

Field	Request (frame 1)	Request (frame 2)
Ethernet destination	ff:ff:ff:ff:ff:ff (Broadcast)	VMware_0e:d1:ce (your MAC — unicast)
ARP opcode	1	2
Sender protocol address	192.168.159.128 (you)	192.168.159.2 (gateway)
Sender hardware address	VMware_0e:d1:ce (you)	00:50:56:f2:0e:73 (gateway)
Target protocol address	192.168.159.2 (gateway)	192.168.159.128 (you)
Forensic interpretation	"Who has 192.168.159.2?"	"192.168.159.2 is at 00:50:56:f2:0e:73"

Part C

```
(kali㉿kali)-[~/SBT-DF203-Lab5]
$ PCAP=working/arp_working.pcap
tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields \
-e frame.number -e frame.time_epoch -e eth.src -e eth.dst \
-e arp.src.proto_ipv4 -e arp.src.hw_mac -e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
| tee reports/arp_replies.tsv
4      1678591364.904123642      00:50:56:86:02:65      00:50:56:86:cb:fc      136.160.
215.194 00:50:56:86:02:65      136.160.215.15 00:50:56:86:cb:fc
6      1678591369.943043799      00:50:56:86:cb:fc      00:50:56:86:02:65      136.160.
215.15  00:50:56:86:cb:fc      136.160.215.194 00:50:56:86:02:65
```

```
(kali㉿kali)-[~/SBT-DF203-Lab5]
$ tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields -e arp.src.proto_ipv4 -e arp.src.hw_m
ac \
| sort | uniq -c | sort -nr | tee reports/ip_mac_claims.txt
1 136.160.215.194 00:50:56:86:02:65
1 136.160.215.15  00:50:56:86:cb:fc

(kali㉿kali)-[~/SBT-DF203-Lab5]
$ tshark -r working/arp_working.pcap -Y 'arp.opcode==1 && eth.dst!=ff:ff:ff:ff:ff:ff'
5 46.883509791 VMware_86:02:65 → VMware_86:cb:fc ARP 60 Who has 136.160.215.15? Tell
136.160.215.194
```

Check	Result
IP-to-MAC conflicts (same IP, different MACs)	None found — each of the 3 IPs (.1, .194, .15) consistently maps to one MAC across all 8 packets
Unsolicited/non-broadcast replies	Both replies (frames 4, 6) are normal unicast responses to preceding requests — expected behavior
Anomalous unicast ARP request	Frame 5 — 136.160.215.194 asks "Who has 136.160.215.15?" sent directly to VMware_86:cb:fc, not broadcast. Violates RFC 826, which requires all ARP requests to be broadcast
Gratuitous-style traffic	Frames 1, 7 — periodic self-announcements from the gateway (.1), 60s apart; frames 2, 8 — probes to unused-looking addresses (.199, .183), consistent with router housekeeping (duplicate-address detection), not spoofing

Part E

Time (+s)	Claimed IP	Claimed MAC	Target	Request Seen First?	Assessment
0.000	136.160.215.1	00:1b:17:00:0a:30	(self-announce, tpa=own IP)	N/A (gratuitous)	Normal — gateway self-announcement
35.133	-----			N/A (probe)	Normal — gateway duplicate-address probe
41.844	-----	-----	136.160.215.194	Yes (broadcast request, frame 3)	Normal — standard resolution
41.845	136.160.215.194	00:50:56:86:02:65	136.160.215.15	Yes, reply to frame 3	Normal — unicast reply to a broadcast request
46.884	-----	-----	136.160.215.1	No — request	Anomalous —

			5	itself sent unicast, not broadcast	RFC 826 violation, targeted/crafted request (frame 5)
46.884	136.160.215.1 5	00:50:56:86:cb :fc	136.160.215.1 94	Yes, reply to frame 5	Normal reply, but responding to an abnormal request
60.000	136.160.215.1	00:1b:17:00:0a :30	(self-announce)	N/A (gratuitous)	Normal — repeats the 60s gateway cycle
63.132	-----	-----	136.160.215.1 83	N/A (probe)	Normal — gateway duplicate-address probe

Restoration and Prevention

```
(kali㉿kali)-[~/SBT-DF203-Lab5]
$ sudo ip neigh flush all
ip neigh show
[sudo] password for kali:

(kali㉿kali)-[~/SBT-DF203-Lab5]
$ ps aux | grep -E '[a]rp.py|[s]capy' | tee reports/process_check.txt

(kali㉿kali)-[~/SBT-DF203-Lab5]
$ ping -c 1 192.168.159.2
ip neigh show | tee reports/arp_table_final.txt
PING 192.168.159.2 (192.168.159.2) 56(84) bytes of data.
64 bytes from 192.168.159.2: icmp_seq=1 ttl=128 time=0.325 ms

— 192.168.159.2 ping statistics —
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.325/0.325/0.325/0.000 ms
192.168.159.2 dev eth0 lladdr 00:50:56:f2:0e:73 DELAY
```

6. ARP-Table Comparison

State	Gateway entry (192.168.159.2)	Source
Initial cache (Step 1, before flush)	00:50:56:f2:0e:73 (STALE)	reports/arp_table_initial.txt
After controlled flush + ping (Part A)	00:50:56:f2:0e:73	reports/arp_table_after_ping.txt
Final, after full restoration (Part 13)	00:50:56:f2:0e:73 (DELAY, re-resolving)	reports/arp_table_final.txt

The initial baseline (Step 1) listed two cached entries — 192.168.159.254 → 00:50:56:ec:66:07 and

192.168.159.2 → 00:50:56:f2:0e:73 — so the gateway's cached MAC was already 00:50:56:f2:0e:73 before

the flush, and it resolved to the identical MAC after the controlled flush-and-ping in Part A, and again after the full flush in Part 13. Across all three checkpoints the gateway's MAC was consistent and no conflicting value was ever observed on this VM's own segment. This confirms the live environment showed no poisoning at any point during the lab session.

7. Impact Assessment

No impact - ARP resolution behaved normally and consistently throughout, and the environment was left in a clean, restored state with no lingering ARP-manipulation process. On the supplied evidence (arp.pcap, Part C): no confirmed man-in-the-middle condition was established, since no IP address was ever claimed by two different MAC addresses in this capture. However, frame 5's unicast (non-broadcast) ARP request is a deviation from standard protocol behaviour that would typically be produced by a crafted packet rather than a normal OS ARP stack. Had this pattern been paired with a following conflicting reply (which is not present in this short 8-packet sample), it would indicate an active MITM attempt against 136.160.215.15 by a host masquerading as, or operating from, 136.160.215.194. As captured, the appropriate conclusion is: a reconnaissance-consistent anomaly was observed, but full poisoning was not conclusively demonstrated within the evidence provided.

8. Restoration and Preventive Controls

Restoration steps executed and verified (see Part 13 screenshot): all learned ARP entries were flushed (ip neigh flush all), no arp.py/scapy process was found running, and the gateway was confirmed to re-resolve correctly to its legitimate MAC (00:50:56:f2:0e:73) after the flush — confirming the analysis VM was returned to a clean, working state.

Recommended preventive controls:

Control	Purpose
Dynamic ARP Inspection (DAI) + DHCP snooping	Automatically validates ARP bindings against snooped DHCP leases on managed switches; would flag frame 5's non-standard unicast request pattern
Static ARP entries for critical infrastructure	Prevents the gateway/router mapping from being overridden by any ARP traffic, solicited or not
Network segmentation and port security	Limits the blast radius of a compromised host attempting ARP manipulation
Encrypted application-layer protocols (TLS/SSH)	Ensures that even if ARP-layer interception succeeds, intercepted

	traffic remains unreadable
ARP monitoring (arpwatch, IDS ARP-spoof rules)	Detects anomalies like non-broadcast requests or IP/MAC conflicts in real time, correlated with switch and endpoint logs

9. Appendix — Chain-of-Custody and Declaration

Field	Entry
Case/lab identifier	SBT-DF203-Lab5-Afeez Bolaji Usman
Trainee name	Afeez Bolaji Usman
Student ID	202514326WY
Date and time started	17-09-2026
Evidence file	arp.pcap
Source	ICDFA-supplied training capture
Original SHA-256	342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e
Working-copy SHA-256	Matches original (verified)
Analysis workstation/VM	Kali Linux (kali@kali)
Live component (Part D)	Not performed — instructor authorization not obtained; supplied evidence and live baseline-only analysis used instead, per manual's permitted scope
Notes on any changes	None — working copy verified byte-identical to original; VM restored to clean ARP state in Part 13